

PES UNIVERSITY

Department of Computer Science and Engineering

UE24CS326: COMPUTER NETWORK SECURITY · ASSIGNMENT 1

Packet Sniffing & Spoofing using Python

STUDENT

Sumedh Girish

INSTRUCTOR

Dr. Preet Kanwal

DATE

August 15, 2026

Setup

The Lab gives a docker compose configuration, which we have to start before any programs can we run. This should give us 3 machines, an `attacker`, `HostA` and `HostB`. This can be done by running

```
docker compose up
```

in the `Labsetup` directory.

We begin by installing scapy, which is a python library that can be used to sniff and spoof packets over networks easily. This is done by importing the library as follows.

```
from scapy.all import *
```

Next, we have to choose the Network Interface Card (NIC) we are working with. For this lab, since the program executes inside an attacker docker container, we reference the **bridge interface** of the attacker machine.

```

→ 0s docker compose exec attacker bash
root@hogspspace:/# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eno1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether a8:2b:dd:dc:79:e1 brd ff:ff:ff:ff:ff:ff
    altname enp3s0
    altnamename enxa82bdddcc79e1
3: wlp4s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 14:b5:cd:3c:77:07 brd ff:ff:ff:ff:ff:ff
    altnamename wlx14b5cd3c7707
    inet 10.20.203.202/21 brd 10.20.207.255 scope global dynamic noprefixroute wlp4s0
        valid_lft 155sec preferred_lft 155sec
    inet6 fe80::81d1:7056:232b:c18b/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
4: tailscale0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1280 qdisc fq_codel state UNKNOWN group default qlen 500
    link/none
    inet 100.122.145.26/32 scope global tailscale0
        valid_lft forever preferred_lft forever
    inet6 fd7a:115c:a1e8::db35:911b/128 scope global
        valid_lft forever preferred_lft forever
    inet6 fe80::45ba:c249:8bcf:3c32/64 scope link stable-privacy
        valid_lft forever preferred_lft forever
5: ztyjkfaexz: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 2800 qdisc fq_codel state UNKNOWN group default qlen 1000
    link/ether 1a:4a:cc:70:9f:db brd ff:ff:ff:ff:ff:ff
    inet 192.168.191.16/24 brd 192.168.191.255 scope global ztyjkfaexz
        valid_lft forever preferred_lft forever
    inet 192.168.191.144/24 brd 192.168.191.255 scope global secondary ztyjkfaexz
        valid_lft forever preferred_lft forever
    inet6 fe80::184a:ccff:fe70:9fdb/64 scope link
        valid_lft forever preferred_lft forever
6: br-68dca9f86238: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 0a:4a:bf:a3:63:c4 brd ff:ff:ff:ff:ff:ff
    inet 10.9.0.1/24 brd 10.9.0.255 scope global br-68dca9f86238
        valid_lft forever preferred_lft forever
    inet6 fe80::84a:bfff:fea3:63c4/64 scope link
        valid_lft forever preferred_lft forever
7: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 7a:7a:54:9c:11:f5 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
8: veth0df0d4@i42: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-68dca9f86238 state UP group default
    link/ether f6:fc:68:ac:2b:7a brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::f4fc:68ff:feac:2b7a/64 scope link
        valid_lft forever preferred_lft forever
9: veth96121c@i42: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-68dca9f86238 state UP group default

```

Figure 1: Finding attacker NIC

```
BRIDGE_INTERFACE = "br-68dca9f86238"
```

Finally, to segregate between subsection solutions, we create a class called `Task1` which will contain all code for this assignment.

Question 1 · Sniffing Packets

The objective of this task is to learn how to use Scapy to do packet sniffing in Python programs.

SOLUTION

Task 1A

```
@staticmethod
def _1A():
    def capture(pkt):
        print(pkt.summary(), flush=True)

    _ = sniff(iface=BRIDGE_INTERFACE, prn=capture)
```

This code tries to print all packets it sees over the interface. This is done using the `sniff(...)` function exposed by the scapy library. The `iface` parameter refers to the target NIC and the `prn` parameter allows specifying a callback function which is called every time a packet arrives at the interface.

Task 1B

Next we look towards segregating the output to only packets we are interested in. While this can be written with vanilla python code, the magnitude of packets can make the program slow. Thus scapy allows implicitly performing this operation using **packet filters**.

These packet filters are compiled and applied by the kernel directly onto the NIC driver configuration — making the filtering extremely efficient.

```
@staticmethod
def _1B():
    def capture(pkt):
        print(pkt.summary(), flush=True)

    filterA = "icmp"
    filterB = "tcp and src host 10.9.0.5 and dst port 23"
    filterC = "net 10.9.0.0/16"

    _ = sniff(iface=BRIDGE_INTERFACE, filter=filterA, prn=capture)
```

The above code demonstrates a few different types of filters in action. We apply filters by passing the packet filter code as a string into the `filter` option in `sniff(...)`.

1. The `filterA` accepts all ICMP packets
2. The `filterB` accepts all TCP packets from the IP address `10.9.0.5` sent to port `23`
3. The `filterC` accepts all packets in the subnet `10.9.0.0/16`

Question 2 · Spoofing Packets

The objective of this task is to spoof IP packets with an arbitrary source IP address. We will spoof ICMP echo request packets and send them to another VM on the same network.

SOLUTION

Task 2A

We can also use Scapy to create packets with arbitrary data. This is called **packet spoofing**.

```
@staticmethod
def _2A():
    pkt = IP(src="10.9.0.1", dst="10.9.0.5") / ICMP()
    send(pkt, verbose=False)
```

The following code creates the simplest possible ICMP packet. Scapy allows creating packets in individual layers. Each layer is separated with a `/` where the components are initialized using constructors of the corresponding protocol type.

Here we explicitly define an ICMP packet wrapped by a custom IP packet. The other layers are automatically inferred by scapy if not provided.

The `send(...)` command sends a constructed packet via any available interface that satisfies the parameters specified in the packet.

Task 2B

More importantly, we can create packets with completely nonsensical or false information.

```
@staticmethod
def _2B():
    pkt = IP(src="1.2.3.4", dst="5.6.7.8")
    send(pkt, verbose=False)
```

Here the IP address `1.2.3.4` does not exist on the network, but is still sent via the interface successfully.

Question 3 · Traceroute

The objective of this task is to implement a simple traceroute tool using Scapy to estimate the distance, in terms of number of routers, between your VM and a selected destination.

SOLUTION

Traceroute works by manipulating the **Time To Live(TTL)** attribute of an IP packet to discover routers along a the path to a given destination one step at a time.

We start by looking at routers exactly 1 hop away, and increment TTL either until we reach our destination or there is an error. The code initialization for this is as follows.

```
def _3(target):  
    ttl = 1  
    while True:  
        ...
```

Next we want to construct an ICMP packet that is headed to the given target. The catch is that we only give utmost as long as the value of `ttl` to live. So on the first iteration, this value is 1.

```
pkt = IP(dst=target, ttl=ttl) / ICMP()  
reply = sr1(pkt, verbose=False, timeout=2)
```

Here we see a new Scapy function `sr1(...)`. This function sends a given packet once and **waits for its corresponding reply**. As a router may choose not to inform of the packet being dropped or lost, we supply a `timeout` parameter set to 2 seconds.

```
if not reply:  
    print(f"{ttl:3} hops: * * *", flush=True)  
    ttl += 1  
    continue
```

In the case that a router did choose not to reply, we have to hope that the next router in the chain does reply. Thus we increment `ttl` and continue.

```
print(f"{ttl:3} hops: {reply[IP].src}", flush=True)  
if reply[ICMP].type == 0:  
    print(f"Done. Terminated at {reply[IP].src}.", flush=True)  
    break  
elif reply[ICMP].type == 11:  
    ttl += 1  
else:  
    print(  
        f"Stopped by ICMP Type {reply[ICMP].type} at {reply[IP].src}.",  
        flush=True,  
    )  
    break
```

When we do get a reply, we need to test if the reply was because the packet was dropped midway, in which case we increment `ttl` and resend, or whether it successfully reached the destination when we terminate and exit.

Question 4 · Sniffing and-then Spoofing

We want to write a program that replies to all ICMP echo requests on the network regardless of its availability

SOLUTION

This exercise can be done by reversing the source and destination of the input packet, and changing its ICMP type before sending it back. The code(which is mostly self explanatory) is shown below

```
@staticmethod
def _4():
    def spoof_ICMP(pkt):
        if IP in pkt and ICMP in pkt and pkt[ICMP].type == 8:
            print(f"Spoofted packet from {pkt[IP].src}", end=" | ")

            pkt = (
                IP(src=pkt[IP].dst, dst=pkt[IP].src, ihl=pkt[IP].ihl)
                / ICMP(type=0, id=pkt[ICMP].id, seq=pkt[ICMP].seq)
                / pkt[ICMP].load
            )

            print(f"Sending {pkt.summary()}", flush=True)
            send(pkt, verbose=False)

    _ = sniff(iface=BRIDGE_INTERFACE, prn=spoof_ICMP)
```

We simply ricochet any packets we receive that are ICMP echo requests right back with the same data as replies.

Python Code Summary

```
from scapy.all import *
from scapy.layers.inet import ICMP, IP

BRIDGE_INTERFACE = "br-68dca9f86238"

class Task1:
    @staticmethod
    def _1A():
        def capture(pkt):
            print(pkt.summary(), flush=True)

        _ = sniff(iface=BRIDGE_INTERFACE, prn=capture)

    @staticmethod
    def _1B():
        def capture(pkt):
            print(pkt.summary(), flush=True)

        filterA = "icmp"
        filterB = "tcp and src host 10.9.0.5 and dst port 23"
        filterC = "net 10.9.0.0/16"

        _ = sniff(iface=BRIDGE_INTERFACE, filter=filterA, prn=capture)

    @staticmethod
    def _2A():
        pkt = IP(src="10.9.0.1", dst="10.9.0.5")
        send(pkt, verbose=False)

    @staticmethod
    def _2B():
        pkt = IP(src="1.2.3.4", dst="5.6.7.8")
        send(pkt, verbose=False)

    @staticmethod
    def _3(target):
        ttl = 1
        while True:
            pkt = IP(dst=target, ttl=ttl) / ICMP()
            reply = sr1(pkt, verbose=False, timeout=2)

            if not reply:
                print(f"{ttl:3} hops: * * *", flush=True)
                ttl += 1
                continue

            print(f"{ttl:3} hops: {reply[IP].src}", flush=True)
            if reply[ICMP].type == 0:
                print(f"Done. Terminated at {reply[IP].src}.", flush=True)
                break
            elif reply[ICMP].type == 11:
```

```
        ttl += 1
    else:
        print(
            f"Stopped by unexpected ICMP Type {reply[ICMP].type}",
            flush=True,
        )
        break

@staticmethod
def _4():
    def spoof_ICMP(pkt):
        if IP in pkt and ICMP in pkt and pkt[ICMP].type == 8:
            print(f"Spoofted packet from {pkt[IP].src}", end=" | ")

            pkt = (
                IP(src=pkt[IP].dst, dst=pkt[IP].src, ihl=pkt[IP].ihl)
                / ICMP(type=0, id=pkt[ICMP].id, seq=pkt[ICMP].seq)
                / pkt[ICMP].load
            )

            print(f"Sending {pkt.summary()}", flush=True)
            send(pkt, verbose=False)

    _ = sniff(iface=BRIDGE_INTERFACE, prn=spoof_ICMP)
```

Setup



```
- Bs docker compose up
[+] up 4/4
✓ Network net-10.9.0.0 Created 0.1s
✓ Container seed-attacker Created 0.1s
✓ Container hostB-10.9.0.6 Created 0.1s
✓ Container hostA-10.9.0.5 Created 0.1s
Attaching to hostA-10.9.0.5, hostB-10.9.0.6, seed-attacker
hostA-10.9.0.5 | * Starting internet superserver inetd [ OK ]
hostB-10.9.0.6 | * Starting internet superserver inetd [ OK ]
```

Enable Watch Detach

Figure 2: `docker compose up` output

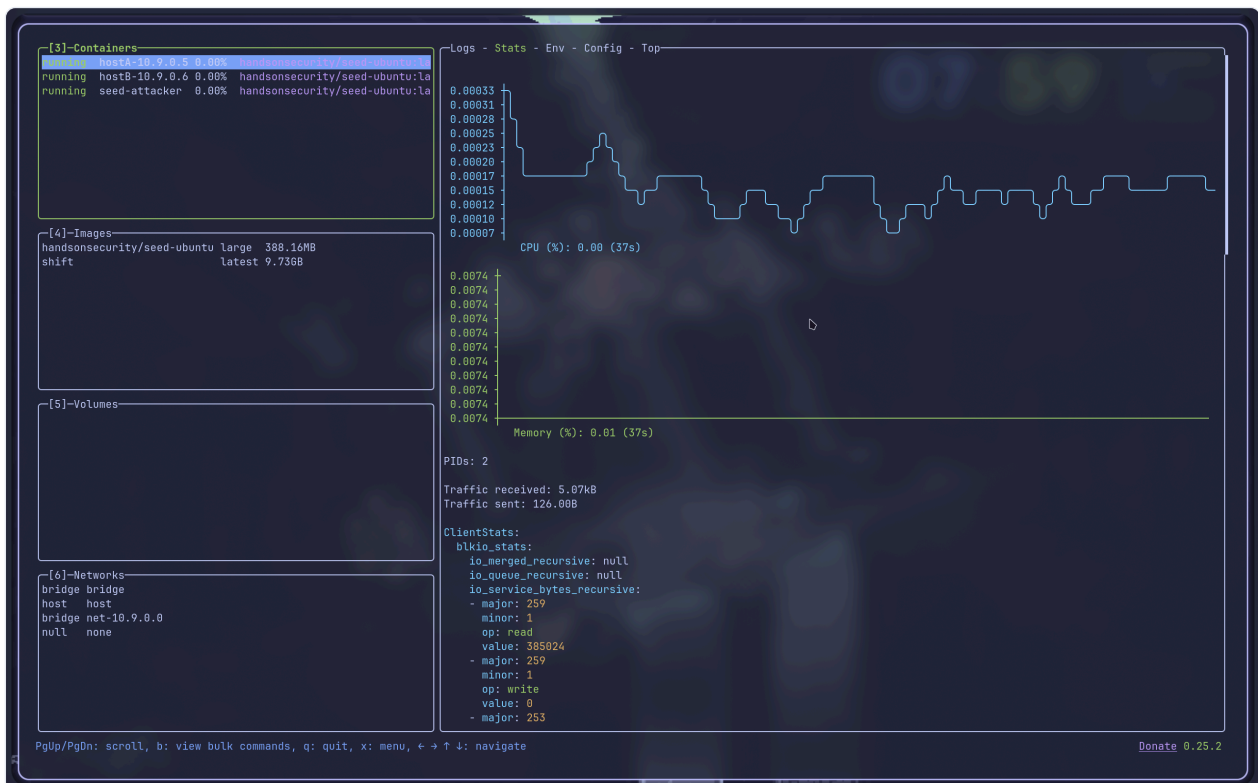


Figure 3: Running container status

```

- 2m19s ○ docker compose exec -T attacker python3 < --code/task
s.py
Ether / IP / ICMP 10.9.0.5 > 142.251.222.78 echo-request 0 / Raw
Ether / IP / ICMP 142.251.222.78 > 10.9.0.5 echo-reply 0 / Raw
Ether / IP / ICMP 10.9.0.5 > 142.251.222.78 echo-request 0 / Raw
Ether / ARP who has 100.71.229.125 says 10.9.0.1
Ether / ARP who has 172.17.0.1 says 10.9.0.1
Ether / ARP who has 172.18.0.1 says 10.9.0.1
Ether / ARP who has 172.19.0.1 says 10.9.0.1
Ether / ARP who has 172.20.0.1 says 10.9.0.1
Ether / ARP who has 172.28.0.1 says 10.9.0.1
Ether / ARP who has 157.50.167.188 says 10.9.0.1
Ether / IP / ICMP 142.251.222.78 > 10.9.0.5 echo-reply 0 / Raw
Ether / ARP who has 157.50.167.188 says 10.9.0.1
Ether / ARP who has 172.20.0.1 says 10.9.0.1
Ether / ARP who has 172.19.0.1 says 10.9.0.1
Ether / ARP who has 172.18.0.1 says 10.9.0.1
Ether / ARP who has 172.17.0.1 says 10.9.0.1
Ether / ARP who has 100.71.229.125 says 10.9.0.1
Ether / ARP who has 157.50.167.188 says 10.9.0.1
Ether / ARP who has 100.71.229.125 says 10.9.0.1
Ether / ARP who has 172.17.0.1 says 10.9.0.1
Ether / ARP who has 172.18.0.1 says 10.9.0.1
Ether / ARP who has 172.19.0.1 says 10.9.0.1
Ether / ARP who has 172.20.0.1 says 10.9.0.1
Ether / ARP who has 10.9.0.5 says 10.9.0.1
Ether / ARP who has 10.9.0.1 says 10.9.0.5
Ether / ARP is at c2:1f:ad:13:51:1d says 10.9.0.1
Ether / ARP is at 2e:06:e5:b3:4e:0f says 10.9.0.5
Ether / ARP who has 10.9.0.5 says 10.9.0.6
Ether / ARP is at 2e:06:e5:b3:4e:0f says 10.9.0.5
Ether / IP / TCP 10.9.0.6:52624 > 10.9.0.5:9090 S
Ether / IP / TCP 10.9.0.5:9090 > 10.9.0.6:52624 SA
Ether / IP / TCP 10.9.0.6:52624 > 10.9.0.5:9090 A
Ether / ARP who has 10.9.0.6 says 10.9.0.5
Ether / ARP is at 1a:03:c3:88:80:51 says 10.9.0.6
Ether / IP / TCP 10.9.0.6:52624 > 10.9.0.5:9090 PA / Raw
Ether / IP / TCP 10.9.0.5:9090 > 10.9.0.6:52624 A
Ether / IP / TCP 10.9.0.5:9090 > 10.9.0.6:52624 PA / Raw
Ether / IP / TCP 10.9.0.6:52624 > 10.9.0.5:9090 A
|

root@906ea8cca951:/# ping -c 2 google.com
PING google.com (142.251.222.78) 56(84) bytes of data.
64 bytes from pnbomb-bp-in-f14.1e100.net (142.251.222.78)
: icmp_seq=1 ttl=118 time=19.8 ms
64 bytes from pnbomb-bp-in-f14.1e100.net (142.251.222.78)
: icmp_seq=2 ttl=118 time=24.4 ms

--- google.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1
002ms
rtt min/avg/max/mdev = 19.794/22.078/24.362/2.284 ms
root@906ea8cca951:/# nc -lnvp 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.6 52624
hi
hello
|

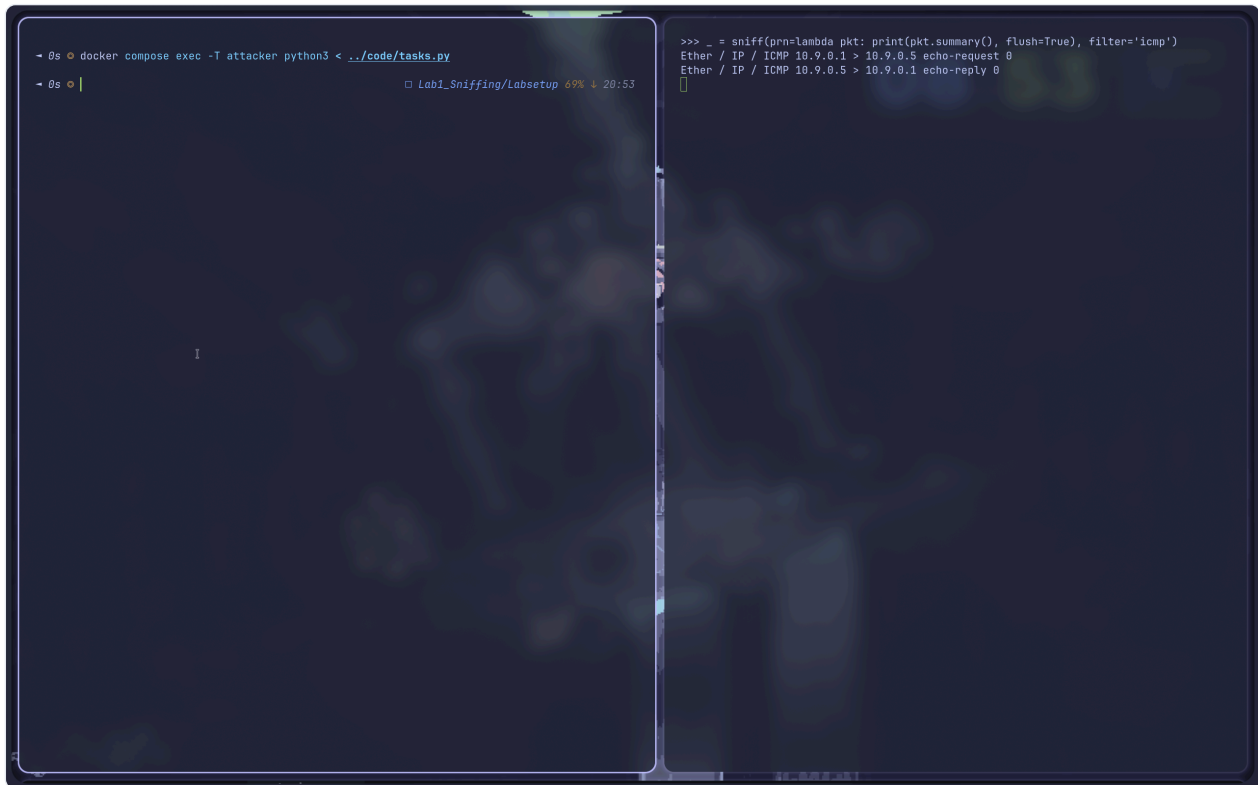
root@b51c58224a62:/# nc 10.9.0.5 9090
hi
hello
|

```

```
- tm308 ○ docker compose exec -T attacker python3 < ../code/task  
s.py  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet S  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet A  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet PA / Raw  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet A  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet PA / Raw  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet A  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet PA / Raw  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet A  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet PA / Raw  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet A  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet PA / Raw  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet A  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet PA / Raw  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet A  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet PA / Raw  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet A  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet PA / Raw  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet A  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet PA / Raw  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet A  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet PA / Raw  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet A  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet PA / Raw  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet A  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet PA / Raw  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet A  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet PA / Raw  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet A  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet PA / Raw  
Ether / IP / TCP 10.9.0.5:51146 > 10.9.0.6:telnet A
```

```
root@96e6a8cca951:/# telnet 10.9.0.6  
Trying 10.9.0.6...  
Connected to 10.9.0.6.  
Escape character is '^['.  
Ubuntu 20.04.1 LTS  
b51c58224a62 login: root  
Password:  
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 7.1.8-zen1-3-zen  
x86_64)  
  
 * Documentation:  https://help.ubuntu.com  
 * Management:    https://landscape.canonical.com  
 * Support:       https://ubuntu.com/advantage  
  
This system has been minimized by removing packages and c  
ontent that are  
not required on a system that users do not log into.  
  
To restore this content, you can run the 'unminimize' com  
mand.  
  
The programs included with the Ubuntu system are free sof  
tware;  
the exact distribution terms for each program are describ  
ed in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent p  
ermitted by  
applicable law.  
  
root@b51c58224a62:~#
```

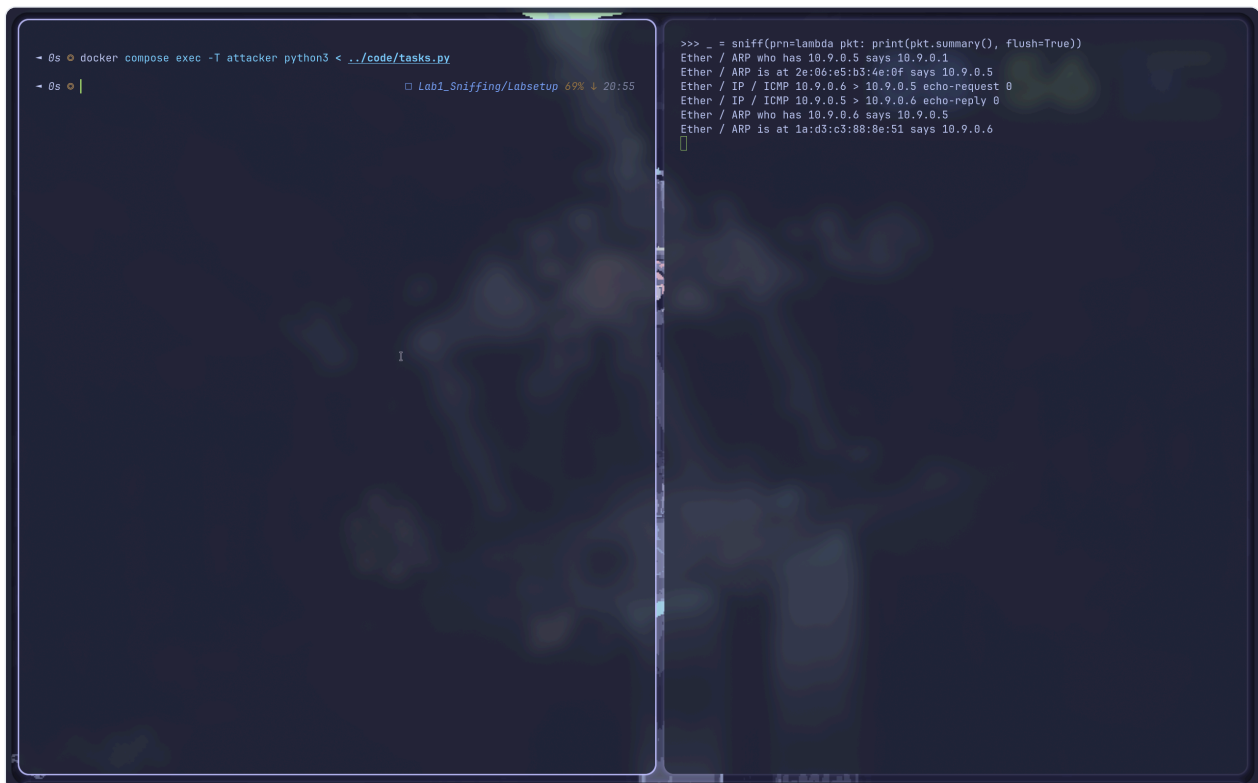
10 / 12



The screenshot shows a terminal window with two panes. The left pane displays the command `docker compose exec -T attacker python3 < ../code/tasks.py` and the output `Lab1_Sniffing/Labsetup 69% 20:53`. The right pane shows the output of a Python script using `sniff` to capture ICMP traffic. The output is as follows:

```
>>> _ = sniff(prn=lambda pkt: print(pkt.summary(), flush=True), filter='icmp')
Ether / IP / ICMP 10.9.0.1 > 10.9.0.5 echo-request 0
Ether / IP / ICMP 10.9.0.5 > 10.9.0.1 echo-reply 0
```

Figure 6: Task 2A



The screenshot shows a terminal window with two panes. The left pane displays the command `docker compose exec -T attacker python3 < ../code/tasks.py` and the output `Lab1_Sniffing/Labsetup 69% 20:55`. The right pane shows the output of a Python script using `sniff` to capture ARP and ICMP traffic. The output is as follows:

```
>>> _ = sniff(prn=lambda pkt: print(pkt.summary(), flush=True))
Ether / ARP who has 10.9.0.5 says 10.9.0.1
Ether / ARP is at 2e:06:e5:b3:4e:0f says 10.9.0.5
Ether / IP / ICMP 10.9.0.6 > 10.9.0.5 echo-request 0
Ether / IP / ICMP 10.9.0.5 > 10.9.0.6 echo-reply 0
Ether / ARP who has 10.9.0.6 says 10.9.0.5
Ether / ARP is at 1a:d3:c3:88:8e:51 says 10.9.0.6
```

Figure 7: Task 2B

```

- 14s docker compose exec -T attacker python3 < ../code/tasks.py
Traceroute: Google
1 hops: 10.81.234.127
2 hops: * * *
3 hops: 10.40.12.141
4 hops: 10.40.9.93
5 hops: 125.22.44.73
6 hops: * * *
7 hops: 72.14.197.10
8 hops: 142.250.230.197
9 hops: 142.250.235.107
10 hops: 8.8.8.8
Done. Terminated at 8.8.8.8.

Traceroute: HostA
1 hops: 10.9.0.5
Done. Terminated at 10.9.0.5.

Traceroute: Cloudflare
1 hops: 10.81.234.127
2 hops: * * *
3 hops: 10.40.12.141
4 hops: 10.40.9.93
5 hops: 125.22.44.73
6 hops: 116.119.50.87
7 hops: * * *
8 hops: 162.158.52.19
9 hops: 1.1.1.1
Done. Terminated at 1.1.1.1.

- 11s

```

Figure 8: Task 3

```

- 2s docker compose exec -T attacker python3 < ../code/tasks.py
Spoofed packet from 10.9.0.5 | Sending IP / ICMP 1.2.3.4 > 10.9.0.5 echo-reply 0 / Raw
Spoofed packet from 10.9.0.5 | Sending IP / ICMP 1.2.3.4 > 10.9.0.5 echo-reply 0 / Raw
Spoofed packet from 10.9.0.5 | Sending IP / ICMP 1.2.3.4 > 10.9.0.5 echo-reply 0 / Raw
Spoofed packet from 10.9.0.5 | Sending IP / ICMP 1.2.3.4 > 10.9.0.5 echo-reply 0 / Raw
Spoofed packet from 10.9.0.5 | Sending IP / ICMP 1.2.3.4 > 10.9.0.5 echo-reply 0 / Raw

root@996e6a8cca951:/# ping 1.2.3.4 -c 5
PING 1.2.3.4 (1.2.3.4) 56(84) bytes of data.
--- 1.2.3.4 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4079ms

root@996e6a8cca951:/# ping 1.2.3.4 -c 5
PING 1.2.3.4 (1.2.3.4) 56(84) bytes of data.
64 bytes from 1.2.3.4: icmp_seq=1 ttl=64 time=34.5 ms
64 bytes from 1.2.3.4: icmp_seq=2 ttl=64 time=6.60 ms
64 bytes from 1.2.3.4: icmp_seq=3 ttl=64 time=8.87 ms
64 bytes from 1.2.3.4: icmp_seq=4 ttl=64 time=14.6 ms
64 bytes from 1.2.3.4: icmp_seq=5 ttl=64 time=9.81 ms
--- 1.2.3.4 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4009ms
rtt min/avg/max/mdev = 6.603/14.867/34.498/10.151 ms
root@996e6a8cca951:/#

```

Figure 9: Task 4